R - A to Z Data Analyst Guide

R basics, tidyverse, dplyr, tidyr, ggplot2, statistics and interview preparation

Simple English + easy explanation + examples + interview preparation

How to use this guide: Read one chapter -> type the example yourself -> do the practice task -> answer the interview question aloud.

Contents / Learning Order

1. **R Foundations for Data Analysis** - Understand R/RStudio, objects and basic syntax
2. **R Data Structures** - Know vectors, matrices, lists, factors and data frames
3. **Conditions, Loops and Functions** - Write reusable R logic
4. **Packages and Tidyverse** - Use modern data-analysis packages
5. **Importing and Inspecting Data** - Read files and understand structure
6. **dplyr - Select, Filter and Arrange** - Get needed rows and columns
7. **dplyr - Mutate, Summarise and Group By** - Create columns and KPI summaries
8. **Cleaning Missing and Duplicate Data** - Prepare reliable analysis data
9. **Joins in dplyr** - Combine related tables
10. **Reshaping with tidyr** - Convert wide and long formats
11. **Dates with lubridate** - Create reporting periods
12. **Visualization with ggplot2** - Build clear analytical charts
13. **Statistics and Modeling Basics in R** - Use R strengths for analysis
14. **Export, Reporting and SQL Connectivity** - Deliver analysis outputs
15. **R End-to-End Analyst Project** - Analyze a complete business dataset

How this tool fits in a Data Analyst job

Job workflow: Raw data -> tidyverse cleaning -> analysis/statistics -> ggplot/reporting -> Insight

1. R Foundations for Data Analysis

R and RStudio workflow

Easy meaning: R is a programming language focused on statistics, data analysis and visualization. RStudio is a common IDE for writing and running R code.

Step-by-step:

16. Create a project/script.
17. Install/load packages.
18. Import data.
19. Inspect and clean.
20. Analyze.
21. Visualize.
22. Export/report.

Interview question: Why use R for data analysis?

Objects and basic types

- numeric
- integer
- character
- logical
- factor (categorical)
- Date/POSIXct for dates/times

```
name <- "Koushik"
age <- 22
salary <- 35000.5
is_active <- TRUE
class(salary)
```

Operators

```
a <- 10
b <- 3
a + b
a / b
a %% b
a^b
a > b
a == b
```

Practice task: Create Sales and Profit objects and calculate margin.

2. R Data Structures

Vectors

Easy meaning: A vector stores values of a common basic type.

```
sales <- c(1000, 1200, 900, 1500)
mean(sales)
sales[sales > 1000]
```

Matrix and list

Where/why we use it: Matrices are 2D same-type data; lists can contain mixed objects.

```
m <- matrix(1:6, nrow=2)
info <- list(name="Koushik", skills=c("SQL","Power BI"), exp=0)
```

Factors

Easy meaning: Factors represent categorical data and can have ordered levels.

```
region <- factor(c("South","North","South"))
levels(region)
```

Data frame

```
df <- data.frame(Product=c("A","B"), Sales=c(1000,2000))
str(df)
```

Practice task: Create a 5-row sales data frame with Product, Region, Sales and Profit.

3. Conditions, Loops and Functions

if / else

```
sales <- 12000
if (sales >= 50000) {
  band <- "High"
} else if (sales >= 10000) {
  band <- "Medium"
} else {
  band <- "Low"
}
```

for loop and vectorization

- R is naturally vectorized.
- Prefer vectorized/tidyverse operations over unnecessary loops for data tasks.

```
for (x in c(1,2,3)) {
  print(x)
}
```

```
# vectorized
c(1,2,3) * 10
```

Functions

```
profit_margin <- function(sales, profit) {
  if (sales == 0) return(0)
  profit / sales
}
profit_margin(10000, 2000)
```

Interview question: How do you create and call a function in R?

4. Packages and Tidyverse

Install and load packages

- readr: import text data
- dplyr: transform data
- tidyr: reshape data
- ggplot2: visualization
- stringr: strings
- lubridate: dates

```
install.packages("tidyverse")  # run once
library(tidyverse)             # load each session
```

Pipe operator

Easy meaning: Pipe passes result from one step into next, creating readable workflows.

```
sales_df |>
  filter(Sales > 10000) |>
  arrange(desc(Sales))
```

5. Importing and Inspecting Data

read_csv and read_excel

```
library(readr)
df <- read_csv("sales.csv")
# library(readxl)
# df2 <- read_excel("sales.xlsx")
```

Inspection functions

```
head(df)
glimpse(df)
summary(df)
```

```
dim(df)
colSums(is.na(df))
```

Practice task: Import a dataset and write down rows, columns, types, missing counts and date range.

6. dplyr - Select, Filter and Arrange

select

```
df |> select(OrderID, Region, Sales, Profit)
```

filter

```
df |> filter(Sales > 10000, Region == "South")
```

arrange

```
df |> arrange(desc(Sales))
```

Interview question: What do select, filter and arrange do?

distinct and slice

```
df |> distinct(CustomerID, .keep_all = TRUE)
df |> slice_max(Sales, n = 5)
```

Practice task: Find top 10 South-region orders by Sales.

7. dplyr - Mutate, Summarise and Group By

mutate and case_when

```
df <- df |>
  mutate(ProfitMargin = Profit / Sales,
         SalesBand = case_when(
           Sales >= 50000 ~ "High",
           Sales >= 10000 ~ "Medium",
           TRUE ~ "Low"
         ))
```

group_by + summarise

```
region_summary <- df |>
  group_by(Region) |>
  summarise(
    TotalSales = sum(Sales, na.rm=TRUE),
    TotalProfit = sum(Profit, na.rm=TRUE),
    Orders = n_distinct(OrderID),
```

```
    .groups = "drop"
  )
```

Interview question: What is the purpose of group_by before summarise?

across

```
df |> summarise(across(c(Sales, Profit), ~ mean(.x, na.rm=TRUE)))
```

Practice task: Create Region + Category KPI table.

8. Cleaning Missing and Duplicate Data

Missing values

- Use na.rm=TRUE in many summary functions when appropriate.
- Do not replace missing with zero unless business meaning supports it.

```
colSums(is.na(df))
df |> filter(is.na(Sales))
df <- df |> mutate(Sales = replace_na(Sales, 0))
```

Duplicates

```
df |> count(OrderID) |> filter(n > 1)
df <- df |> distinct(OrderID, .keep_all = TRUE)
```

Text cleanup

```
library(stringr)
df <- df |> mutate(CustomerName = str_to_title(str_trim(CustomerName)))
```

Practice task: Clean blank/missing values, duplicates and inconsistent text case.

9. Joins in dplyr

left_join, inner_join, full_join

Easy meaning: Join syntax is similar to SQL join concepts.

- left_join keeps all left rows
- inner_join keeps matches
- full_join keeps all rows from both tables
- Always check key uniqueness and row count after join

```
result <- orders |>
  left_join(customers, by = "CustomerID")
```

anti_join and semi_join

Where/why we use it: Excellent for data quality and existence checks.

```
orders_without_customer <- orders |> anti_join(customers, by="CustomerID")
customers_with_orders <- customers |> semi_join(orders, by="CustomerID")
```

Interview question: What is anti_join used for?

10. Reshaping with tidyr

pivot_longer

```
long <- wide |>
  pivot_longer(cols=Jan:Dec, names_to="Month", values_to="Sales")
```

pivot_wider

```
wide <- long |>
  pivot_wider(names_from=Month, values_from=Sales)
```

Practice task: Convert monthly columns Jan-Dec into Month/Sales rows.

11. Dates with lubridate

Parse and extract dates

Where/why we use it: Makes date parsing and extraction easier.

```
library(lubridate)
df <- df |> mutate(
  OrderDate = ymd(OrderDate),
  Year = year(OrderDate),
  Month = month(OrderDate, label=TRUE),
  Quarter = quarter(OrderDate)
)
```

Date intervals

```
df <- df |> mutate(ShippingDays = as.numeric(ShipDate - OrderDate))
```

Practice task: Create Year, Month, Quarter and ShippingDays.

12. Visualization with ggplot2

ggplot grammar

Easy meaning: ggplot2 builds charts from data + aesthetics + geometries + scales/themes.

```
library(ggplot2)
ggplot(region_summary, aes(x=Region, y=TotalSales)) +
  geom_col() +
  labs(title="Sales by Region", x="Region", y="Sales")
```

Line and scatter

```
ggplot(monthly, aes(x=Month, y=Sales, group=1)) + geom_line()

ggplot(df, aes(x=Sales, y=Profit)) + geom_point()
```

Practice task: Create region bar, monthly line, sales-profit scatter and histogram.

13. Statistics and Modeling Basics in R

Descriptive statistics

```
mean(df$Sales, na.rm=TRUE)
median(df$Sales, na.rm=TRUE)
sd(df$Sales, na.rm=TRUE)
quantile(df$Sales, probs=c(.25,.5,.75), na.rm=TRUE)
```

Correlation

```
cor(df$Sales, df$Profit, use="complete.obs")
```

Interview question: Correlation does not prove causation.

Simple linear regression

Easy meaning: Regression estimates relationship between an outcome and predictor variables. For analyst roles, understand interpretation more than memorizing mathematics.

```
model <- lm(Profit ~ Sales + Discount, data=df)
summary(model)
```

Practice task: Fit a simple model and explain coefficient direction, R-squared and p-value at a high level.

14. Export, Reporting and SQL Connectivity

Write files

```
write_csv(region_summary, "region_summary.csv")
# openxlsx/writexl can write Excel files
```

R Markdown / Quarto concept

Easy meaning: Combine code, narrative, tables and plots into reproducible reports.

Where/why we use it: Useful for analysis reports and automated reporting.

Database connectivity concept

Easy meaning: DBI plus a database driver can connect R to SQL databases.

```
# concept
# library(DBI)
# con <- dbConnect(...)
# df <- dbGetQuery(con, "SELECT * FROM Orders")
```

15. R End-to-End Analyst Project

Project workflow

Step-by-step:

23. Import raw sales data.
24. Inspect types/nulls/duplicates.
25. Clean with dplyr/tidyr/stringr/lubridate.
26. Join dimension tables.
27. Create KPIs.
28. Build group summaries.
29. Visualize with ggplot2.
30. Run basic statistical checks.
31. Export summary/report.
32. Explain business insights.

Practice task: Build an R script/report for sales trend, profit margin, top regions, customer segments and discount impact.

Final Revision - Interview Quick Answers

Q: Vector vs list?

A: Vector normally contains one basic type; list can contain mixed object types.

Q: What is a factor?

A: A categorical data type with defined levels.

Q: dplyr use?

A: Data manipulation: select, filter, mutate, summarise, group_by, arrange and joins.

Q: pivot_longer vs pivot_wider?

A: pivot_longer converts wide columns to key-value rows; pivot_wider spreads key values into columns.

Q: ggplot2 concept?

A: Build plots from data, aesthetics and geometric layers.

Q: How handle NA?

A: Use is.na to inspect and choose appropriate removal/replacement; many summaries support na.rm=TRUE.

Q: group_by + summarise?

A: Creates grouped KPI summaries, like SQL GROUP BY.

Q: Why R for analytics?

A: Strong statistics ecosystem, tidy data tools and high-quality reproducible visualization/reporting.

R Mini Project - End-to-End Process

33. Import a sales file with readr.
34. Inspect with glimpse/summary and check NA/duplicates.
35. Clean using dplyr, stringr and lubridate.
36. Join master tables.
37. Create KPIs with mutate/group_by/summarise.
38. Reshape if needed with tidyr.
39. Visualize with ggplot2.
40. Run basic descriptive statistics/correlation.
41. Export summary and create a reproducible report.
42. Explain business insights in interview.

Before Interview - Checklist

- I can explain the tool in simple words.
- I can explain one end-to-end project without reading.
- I typed/practiced the main formulas or syntax myself.
- I know common mistakes and how to validate results.
- I can answer at least 20 core interview questions aloud.
- I can connect the tool with the full analyst workflow: clean -> analyze -> visualize -> insight.

30-Day Practice Suggestion

- Days 1-5: foundations and syntax/UI basics.
- Days 6-12: data cleaning and core analysis topics.
- Days 13-18: advanced calculations/joins/modeling/reshaping.
- Days 19-23: build the mini project from start to finish.
- Days 24-27: repeat project without notes and fix weak areas.
- Days 28-30: interview questions + explain project aloud in simple English.

Remember: Do not only read. Type/click every example yourself. Repetition is what makes the topic stay in memory.

Deep Dive Appendix - R Topics You Must Not Miss

Base R indexing

- Use x[i] for vectors and df[rows, columns] for data frames.
- $ accesses a named column.
- Tidyverse functions are often clearer for business analysis, but base indexing is still important.

apply family awareness

- apply works across matrix/data-frame margins.
- lapply returns list; sapply simplifies result when possible.
- Modern tidyverse purrr provides map-family alternatives.

forcats for factors

- forcats helps reorder and recode factor levels, useful for chart ordering and categorical cleaning.

stringr and regex

- str_detect, str_extract, str_replace and str_split simplify text tasks.

Facets and ggplot layering

- facet_wrap/facet_grid split charts by category.
- Add layers such as points, lines, smoothers, reference lines and annotations carefully.

Hypothesis testing awareness

- t-test compares means under assumptions.
- Chi-square test can test association between categorical variables.
- ANOVA compares means across multiple groups.
- For fresher interviews, explain purpose/assumptions at a high level rather than memorizing formulas.

Reproducible reporting

- Quarto/R Markdown combines narrative, code, tables and charts.
- Projects make file paths and dependencies easier to manage.

Shiny awareness

- Shiny can create interactive web applications from R. It is useful but optional for most entry-level Data Analyst roles.

R Pattern Sheet for Data Analysis

```
library(tidyverse)
library(lubridate)

df <- read_csv("sales.csv")

df_clean <- df |>
  mutate(
    OrderDate = ymd(OrderDate),
    CustomerName = str_to_title(str_trim(CustomerName)),
    ProfitMargin = if_else(Sales != 0, Profit / Sales, 0)
  ) |>
```

```
  distinct(OrderID, .keep_all = TRUE)

summary <- df_clean |>
  group_by(Region) |>
  summarise(
    Sales = sum(Sales, na.rm=TRUE),
    Profit = sum(Profit, na.rm=TRUE),
    Orders = n_distinct(OrderID),
    .groups="drop"
  )
```

R Hands-on Lab - Script/Report Workflow

43. Create an R project and script/report.
44. Load tidyverse/lubridate.
45. Import sales data.
46. Inspect with glimpse/summary.
47. Check NA and duplicate keys.
48. Clean strings/dates/types.
49. Join master tables.
50. Create KPIs with mutate.
51. Summarise with group_by.
52. Reshape with pivot_longer/wider if required.
53. Visualize with ggplot2 and facets.
54. Run descriptive stats/correlation and export results.

30 R Practice Questions / Tasks

- Create vector and filter values.
- Create list and data frame.
- Explain factor.
- Write if/else.
- Write a function.
- Load tidyverse.
- Use pipe.
- Import CSV.
- Use glimpse and summary.
- select columns.
- filter rows.
- arrange descending.
- distinct duplicate IDs.
- mutate calculated field.
- case_when sales bands.
- group_by + summarise.
- Use across.
- Find/handle NA.
- Clean strings with stringr.
- left_join two tables.
- anti_join unmatched rows.

- pivot_longer.
- pivot_wider.
- Parse dates with ymd.
- Calculate date difference.
- Create ggplot bar chart.
- Create line/scatter/facet.
- Calculate mean/median/sd/correlation.
- Explain lm regression at high level.
- Explain R project end-to-end in 2 minutes.

Final One-Page Memory Map

Remember this order: Objects/vectors/data frames -> functions -> tidyverse -> import/profile -> dplyr clean/analyze -> joins/tidyr -> dates -> ggplot -> stats/reporting -> project.